Reviewer Pack — Secant Rank Lower Bound for Disjointness Tensors

Entry 3b212abe5c22 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 06:02:10 UTC

Secant Rank Lower Bound for Disjointness Tensors

Entry ID: `3b212abe5c22` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 06:02:10 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Secant Varieties of Veronese Reembeddings)
Field B (complexity object): Communication Complexity of DISJOINTNESS

Statement:

> For a DISJOINTNESS matrix M ∈ {0,1}^{n×n}, let T_M ∈ R^{n×n×2} be the tensor formed by stacking M and its transpose. The secant rank of T_M is Ω(n).

Rationale (proposer's reasoning):

> Secant rank captures the minimal number of rank-1 tensors needed to span T_M. High secant rank implies geometric complexity, which may correlate with communication complexity via the tensor product structure of DISJOINTNESS.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f58b9e8b079eb36e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `secant varieties Veronese reembeddings disjointness communication complexity` - `tensor rank lower bounds disjointness communication complexity` - `secant rank Veronese reembeddings stacked matrix transpose`

Top relevant hits considered: - [http://arxiv.org/abs/1011.5867v2] Secant Varieties of Segre–Veronese Varieties - [http://arxiv.org/abs/1012.3563v4] Secant varieties to high degree Veronese reembeddings, catalecticant matrices and smoothable Gorenstein schemes - [http://arxiv.org/abs/1503.01099v2] Singularities of secant varieties

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import sys

def generate_disjointness_matrix(n):
    M = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                M[i][j] = 1
                M[j][i] = 1
    return M

def transpose(matrix):
    return [list(row) for row in zip(*matrix)]

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    n = len(B)
    C = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(n):
                C[i][j] += A[i][l] * B[l][j]
    return C

def rank_1_decomposition(T, max_iter=1000, tol=1e-6):
    U = [[random.random() for _ in range(len(T))] for _ in range(len(T[0]))]
    V = [[random.random() for _ in range(len(T[0]))] for _ in range(len(T))]
    for _ in range(max_iter):
        U_new = matrix_multiply(U, T)
        V_new = matrix_multiply(V, transpose(T))
        if max(abs(x) for row in U_new for x in row) < tol and max(abs(x) for row in V_new for x i
            break
        U = [[x / math.sqrt(sum(y**2 for y in row)) for y in row] for row in U_new]
        V = [[x / math.sqrt(sum(y**2 for y in col)) for col in zip(*V_new)] for _ in range(len(V))
    return U, V
```

```
def secant_rank(T):
    U, V = rank_1_decomposition(T)
    rank = 0
    for u_row in U:
        for v_col in zip(*V):
            if sum(x * y for x, y in zip(u_row, v_col)) > tol:
                rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    M = generate_disjointness_matrix(n)
    T_M = [M] + transpose(M)
    secant_rank_value = secant_rank(T_M)
    conjecture_holds = secant_rank_value >= n / 2
    counterexample = "" if conjecture_holds else "secant rank < n/2"
    return {
        "metric_name": "secant_rank",
        "metric_value": secant_rank_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(re
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"secant rank < n/2\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c67fe947.py", line 81, in <module>
```

```
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c67fe947.py", line 66, in run_trial
    secant_rank_value = secant_rank(T_M)
                        ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c67fe947.py", line 53, in secant_rank
    U, V = rank_1_decomposition(T)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c67fe947.py", line 44, in rank_1_decomp
    U_new = matrix_multiply(U, T)
            ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c67fe947.py", line 37, in matrix_multip
    C[i][j] += A[i][l] * B[l][j]
               ~~~~~~~~^~~~~~~~
TypeError: can't multiply sequence by non-int of type 'float'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to a type error in matrix multiplication, preventing any data collection. The pre-registered support condition cannot be evaluated without successful trials. | next: Fix the matrix_multiply function's type handling and re-run tests with multiple seeds to collect empirical evidence.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	37176
2	preregistration	ollama_remote	qwen3:8b	0	0	24205
3	novelty	ollama_remote	qwen3:8b	0	0	15149
4	novelty	ollama_remote	qwen3:8b	0	0	24006
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12921
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10271
7	judge	ollama_remote	qwen3:8b	0	0	20111

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 143839 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/3b212abe5c22.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3b212abe5c22.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3b212abe5c22.tar.gz` (if generated)